Brief History of Logic in AI

Logic is not just a historical artifact.

It has been the persistent backbone of AI, evolving to meet new challenges while remaining central to how we build intelligent systems.

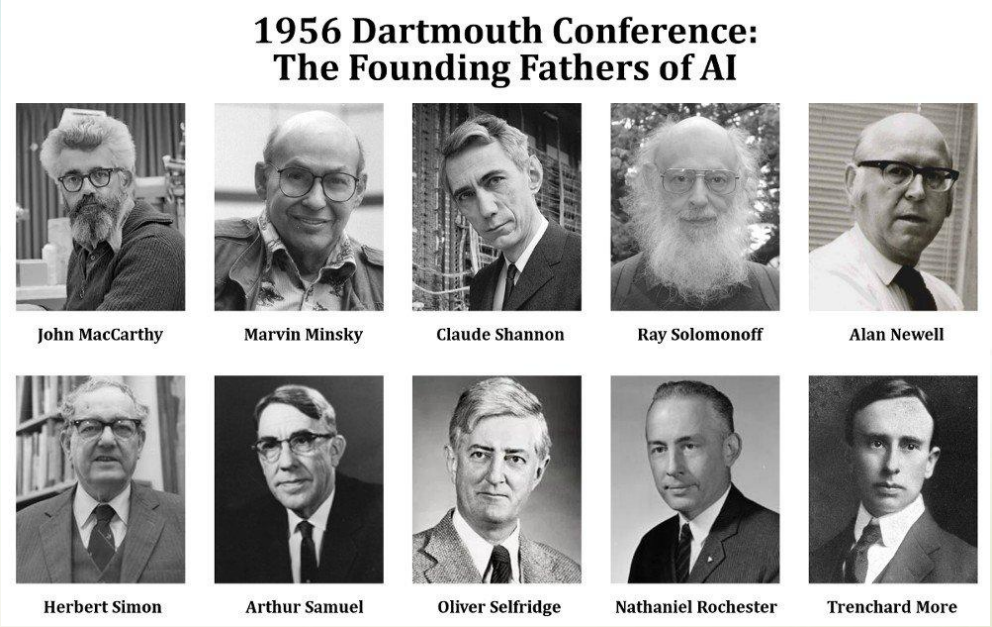
“Persistent backbone of AI”

“Persistent backbone”

Logic & AI: Historical Context

Dartmouth Conference (Summer 1956)

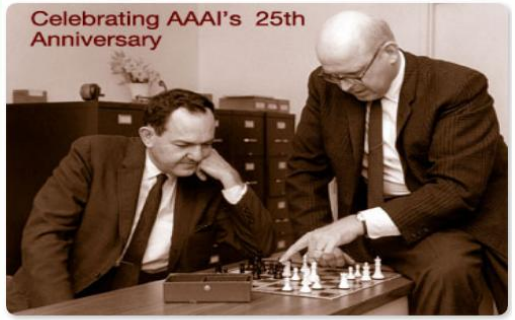
- **Established symbolic reasoning as AI's core approach**: intelligence through manipulation of logical symbols
- **Set the "logic can capture thinking" paradigm** that dominated AI for 30+ years
- **Created the research agenda** focused on formal reasoning, theorem proving, and rule-based systems



Proposal

*"We propose that a 2-month, 10-person study of artificial intelligence be carried out... The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so **precisely described** that a machine can be made to simulate it."*

Key Logic-in-AI Milestones I



Herbert Simon (left) and Allen Newell (right)

Logic Theorist (1956) - Newell & Simon

- First program to prove mathematical theorems using logical reasoning
- Demonstrated that machines could perform formal logical deduction

LISP ("List Processing") (1958) - John McCarthy

- ***Symbolic Processing***: LISP was designed for manipulating symbols rather than numbers, making it ideal for AI tasks like logical reasoning and language understanding.
- ***Code-as-Data (Homoiconicity)***: LISP treats code and data interchangeably, allowing programs to manipulate their own structure—a powerful feature for AI and metaprogramming.
- ***Recursion-Friendly***: LISP supports recursive functions naturally, which is essential for many AI algorithms.
- ***Prefix Notation (Polish Notation)***: Uses fully parenthesized prefix notation, e.g., $(+\ 2\ 3)$ instead of $2\ +\ 3$.
- ***Garbage Collection***: One of the first languages to include automatic memory management.
- ***Extensibility***: Users can define new control structures and language features, making it highly adaptable.
- ***Influence***: It became the dominant language for symbolic AI research for decades.

Key Logic-in-AI Milestones II

Expert Systems Era (1970s-1980s)

- Rule-based systems using "IF-THEN" logical reasoning

Proved logical reasoning could solve real-world problems; established knowledge representation as encoding logical rules; showed logic could capture expert human reasoning

DENDRAL: Stanford (1960s)

- **Purpose**: Assist chemists in identifying molecular structures from mass spectrometry data.
- **Domain-Specific Expert System**: Focused on organic chemistry, particularly structural analysis.
- **Rule-Based Reasoning**: Used encoded ***chemical knowledge*** and inference rules to generate hypotheses.
- **Knowledge Engineering Breakthrough**: Pioneered techniques for capturing expert knowledge in a formal system.
- **Impact**: Demonstrated that expert systems could outperform human experts in narrow domains.

MYCIN: Stanford (1970s)

- **Purpose**: Diagnose bacterial infections and recommend antibiotics.
- **Rule-Based System**: Used hundreds of **"IF-THEN"** rules to simulate expert reasoning.
- **Knowledge Representation**: Encoded ***medical expertise*** as logical rules.
- **Inference Engine**: Applied backward chaining to deduce conclusions from symptoms.
- **Uncertainty Handling**: Introduced **certainty factors** to manage probabilistic reasoning.
- **Impact**: Demonstrated expert-level decision-making in real-world domains like medicine.

Key Logic-in-AI Milestones III

Logic Programming with Prolog (1970s)

- Programming language where you write logical facts and rules, computer 'searches' logical conclusions
- Made logic itself the programming paradigm: showed computation as logical inference
- Influenced AI knowledge representation

```
naiveSort(L1, L2) :- perm(L1, L2), inOrder(L2).
```

```
inOrder([]).
```

Empty list is ordered.

```
inOrder([_]).
```

A list containing 1 element is sorted.

```
inOrder([A, B | T]) :- A =< B, inOrder([B|T]).
```

Check if the first two list elements are ordered.

Check that the tail of the original list is ordered.

```
?- inOrder([1, 2, 3, 4, 5, 6]).
true
```

```
?- inOrder([1, 2, 3, 4, 5, 6, 0]).
false.
```

Key Logic-in-AI Milestones IV

The Logical Reasoning Challenge (1980s Crisis):

- **Problem**: Pure logical systems couldn't handle uncertainty, incomplete information, or common-sense reasoning
- Forced development of fuzzy logic and probabilistic reasoning (expanding what “logic” means in AI)

SAT/SMT Solvers (2000s-present)

- Extremely efficient programs that solve logical satisfiability problems
- Shows logical reasoning can scale to millions of variables; used in software verification, planning, scheduling

SAT: Solving logical boolean formulas

SMT: Boolean logic, but reasoning over integers.

Automated Theorem Proving

- AI systems that prove mathematical theorems using formal logic.
- Demonstrates machine reasoning can surpass humans in formal domains; verifies critical software and hardware

Two Paradigms: Symbolic vs Connectionist Approaches

We have two fundamentally different philosophies about how intelligence works.

Symbolic (Logic-based)

- **"Top-Down" Approach**: Intelligence through manipulation of symbols and rules
- **Explicit knowledge representation:** We tell the computer what we know
- **Rule-based reasoning:** Follows logical steps humans can trace
- **Explainable decisions:** Can precisely explain "why" it reached a conclusion
- **Good for structured domains:** Excels where rules are clear (chess, theorem proving, legal reasoning)

Core assumption: *Intelligence is symbol manipulation*

Connectionist (Neural Networks)

- **"Bottom-Up" Approach**: Intelligence emerges from learning patterns
- **Learning from data:** Discovers patterns we might not see
- **Pattern recognition:** Excels at finding subtle statistical regularities
- **Statistical inference:** Makes decisions based on learned associations
- **Good for perceptual tasks**: Handles messy real-world data (vision, speech, language)

Core assumption: *Intelligence emerges from interconnected simple units*

Reasoning

Explanation

Learning

Adaptation

cat

Integrating paradigms: Neurosymbolic AI

(Recent Trend)

Key Logic-in-AI Milestones V

Neurosymbolic AI (2020s)

- Combining neural networks with logical reasoning (e.g., AlphaGeometry)
- Merges pattern recognition with logical deduction; represents the future where logic remains essential even in the neural age

LLMs and Logical Reasoning

- Large language models showing emergent logical reasoning abilities
- Suggests statistical learning can acquire logical structure; raises questions about whether logic is learned or programmed

LRMs (Large Reasoning Models): designed for complex reasoning, combining symbolic logic with neural networks.

- Unlike LLMs, they focus on deductive, inductive, and mathematical reasoning
- Provide explanations of each step taken facilitating analysis and human verification.

Topics in Logic and Propositional Logic

Introduction to logic

- Arguments, validity, soundness, entailment

Propositional logic

- Sentences, semantics, inference

Inference methods

- Enumeration
- Fundamental rules: Modus ponens
- Forward chaining (data-driven reasoning)
- Backward chaining (goal-driven reasoning)
- Resolution (CNF conversion and resolution rule)
- Soundness and completeness of inference systems

Arguments

Logic is the study of correct reasoning: how to draw ***valid*** conclusions from given information.

Our Challenge: How do we formalize "correct reasoning"?

Before we can automate reasoning, we must understand three key concepts that define what makes reasoning correct.

Definition. An ***argument*** is a set of sentences (premises) and a conclusion.

Premises	All students must have student IDs to enter the library. Sarah entered the library.	If you hit the snooze button more than twice, you will be late for your 8am class. Jake hit the snooze button four times this morning.
Conclusion	Sarah has a student ID.	Jake will be late for his 8am class.

Validity and Soundness

Definition. An argument is ***valid*** if, assuming all its premises are true, the conclusion must also be true.

Definition. An argument is ***sound*** if it is valid *and* all premises are actually true.

Validity is about the **form** of the argument—not whether the premises are actually true. Even an argument with false premises can be valid if the conclusion logically follows.

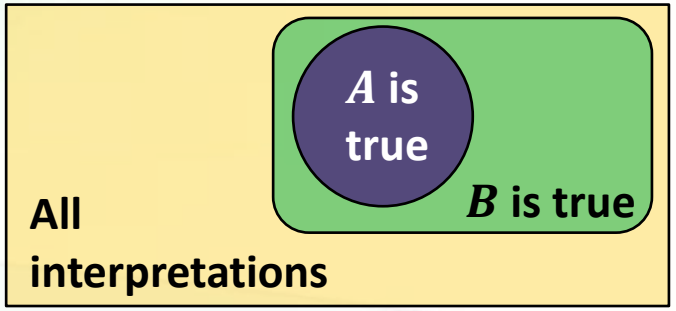
Sound means the conclusion is not only logically guaranteed but also factually correct.

Premises	All smartphones need charging. My phone is a smartphone.	Some students have iPhones. Jessica has an iPhone.	All professors give easy exams. Dr. Alvin is a professor.
Conclusion	My phone needs charging.	Jessica is a student.	Dr. Alvin gives easy exams.
Valid?	Valid	No	Valid
Sound?	Sound	No	No

Entailment

Definition. $A \models B$ if and only if in every interpretation where A is true, B is also true.

B is a **logical consequence** of A; i.e., the truth of A guarantees the truth of B based on logical structure and meaning.



A	It is raining heavily.	Jake hit snooze 4 times and his class starts in 10 minutes.	Sarah received an A on every assignment and every quiz/exam in the course.
B	The ground is wet.	Jake will be late to class.	Sarah will receive an A in the course.
$A \models B$	Yes (Whenever it rains heavily, the ground must be wet.)	No (There is a possible interpretation where A is true but B is false: Jake lives next door and can make it in 2 minutes.)	Yes

Propositional Logic

Propositions

Definition. A ***proposition*** is a phrase that is either True or False (but not both).

Examples. State whether each phrase is a proposition: yes / no.

I have an orange cat.	Yes
$2 + 2 = 4$	Yes
Go directly to jail.	No
It is not true that 3 is an even integer or 7 is a prime number.	Yes
Knock before entering!	No

Propositional Formula

Definition*.** The following strings are valid (well-formed) ***propositional sentences:

Symbols (atomic sentences)

- A ***variable name***: often a capitalized letter. For example, P, Q, R.

Constants

- true, false

Unary Operators

- Negation: $\sim\phi$ where ϕ is a (valid) propositional sentence.

Binary Operators

For ϕ and ψ being (valid) propositional sentences:

Other books/sources may indicate negation as $\neg\phi$ or ϕ'.

- Conjunction: $\phi \wedge \psi$ And
- Disjunction: $\phi \vee \psi$ Or
- Implication: $\phi \Rightarrow \psi$ How we formally state an assumption (ϕ) and statement to prove (ψ).
- Biconditional: $\phi \Leftrightarrow \psi$ How we formally state equivalence.

ϕ: \phi ψ: \psi

Propositional Sentence Examples

Suppose:

- P: "it is cold"
- Q: "it is raining"
- R: "I will wear a jacket"
- S: "It is hot"

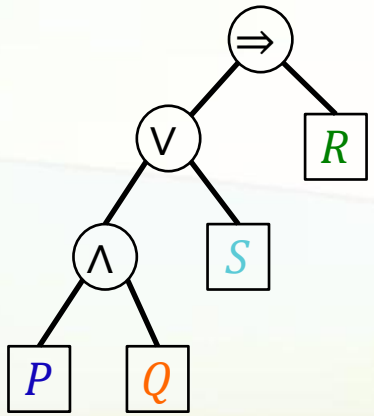
"I will not wear a jacket."	$\sim R$
"If it is cold, then it is raining."	$P \Rightarrow Q$
"If it is raining, then it is cold."	$Q \Rightarrow P$
"If it is cold or it is raining, then I will wear a jacket"	$(P \vee Q) \Rightarrow R$
"I will wear a jacket if and only if it is cold and not hot."	$R \Leftrightarrow (P \wedge \sim S)$

Precedence

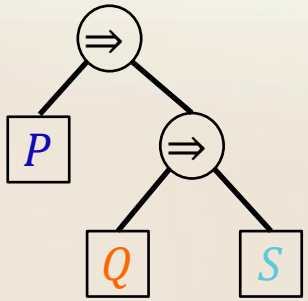
In propositional logic, operator precedence matters. In decreasing order of precedence (i.e, we evaluate in this order):

- Negation (~)
- Conjunction (∧)
- Disjunction (∨)
- Biconditional (⇔) and Conditional (⇒)

$P \wedge Q \vee S \Rightarrow R$



$P \Rightarrow Q \Rightarrow S$

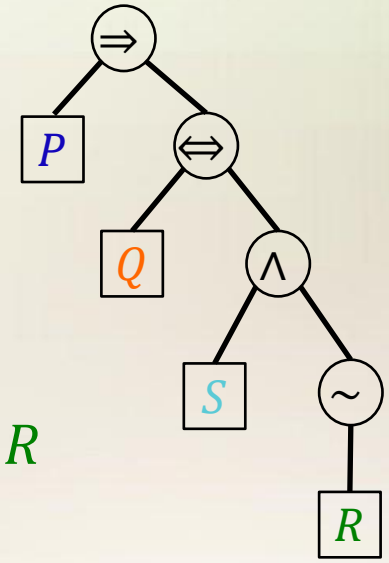


Use parentheses to guarantee well-formedness.

⇒ and ⇔ are right-associative:

$P \Rightarrow Q \Rightarrow S$ is interpreted as $P \Rightarrow (Q \Rightarrow S)$

$P \Rightarrow Q \Leftrightarrow S \wedge \sim R$



Propositional Logic Semantics

Semantics

All of the previous work helped us establish the *syntax of propositional sentences* for our language.

Definition. The ***semantics*** are the compositional rules that define the meaning of logical connectives and how to evaluate the truth of formulas.

Every variable stands for a *primitive proposition* that may be either *true* or *false*.

Primitive Propositions

P : "It is raining."

Q : "My umbrella is open."

R : "I am singing."

S : "I am dancing."

We construct compound sentences containing more than one primitive proposition:

$$\big((P \wedge \sim Q) \wedge (R \vee S)\big)$$

This sentence evaluates to *true* if:

It is raining and my umbrella is *not* open and

furthermore I am singing or dancing (or both).

Models

Let $P \wedge Q$ be a compound propositional sentence.

Consider *configurations* of true / false values for primitives P and Q.

We call each possible configuration of true and false values for a formula a ***model***.

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

There are 4 possible models for $P \wedge Q$.

Definition. Let S be a set of variable names. A ***model*** M is a function that assigns a truth value to every variable name in S. That is,

$$M : S \rightarrow \{True, False\}$$

Definition. A model M ***satisfies*** sentence α if α is true under the assignment of truth values given by M.

A sentence α is **satisfiable** if there exists at least one model that satisfies α (makes the sentence true).

Example

P : "It is raining."

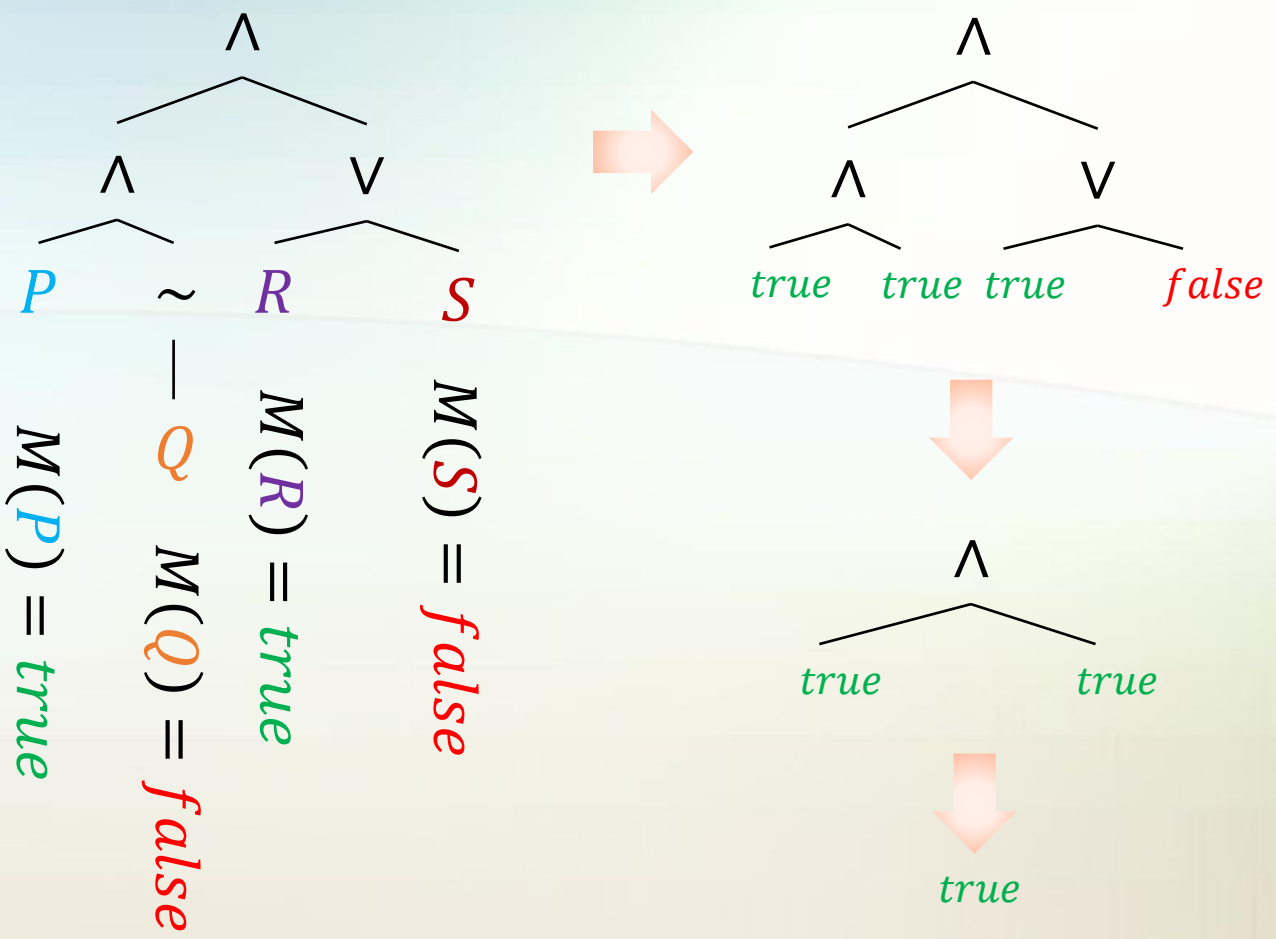
Q : "My umbrella is open."

R : "I am singing."

S : "I am dancing."

$$((P \wedge \sim q) \wedge (R \vee S))$$

It is raining and my umbrella is *not* open and furthermore I am singing or dancing (or both).



We can say that is sentence is satisfiable. (It may evaluate to true for other models.)

Example: Exclusive Or

Exercise. For propositions A and B, the ***exclusive or*** operation, denoted as $A \oplus B$, yields true if and only if exactly one of the operands A or B is true. Define an equivalent logical expression for $A \oplus B$. Demonstrate the equivalence with a truth table.

Solution.

A	B	$A \wedge \sim B$	$\sim A \wedge B$	$(A \wedge \sim B) \vee (\sim A \wedge B)$
T	T	F	F	F
T	F	T	F	T
F	T	F	T	T
F	F	F	F	F

Associativity I

Exercise. For propositions A, B and C, are the operators ∨ and/or ∧ associative operators?

$$(A \vee \mathrm{B}) \vee C \Leftrightarrow A \vee (\mathrm{B} \vee C) \qquad (A \wedge \mathrm{B}) \wedge C \Leftrightarrow A \wedge (\mathrm{B} \wedge C)$$

Solution.

A	B	C	$(A \vee B) \vee C$	$A \vee (B \vee C)$	$(A \wedge B) \wedge C$	$A \wedge (B \wedge C)$
T	T	T	T	T	T	T
T	T	F	T	T	F	F
T	F	T	T	T	F	F
T	F	F	T	T	F	F
F	T	T	T	T	F	F
F	T	F	T	T	F	F
F	F	T	T	T	F	F
F	F	F	F	F	F	F

Clearly, yes. ∧ and ∨ are associative.

Associativity II

Exercise. Is ⇒ an associative operator?

Solution.

A	B	$A \Rightarrow B$	C	$(A \Rightarrow B) \Rightarrow C$	$B \Rightarrow C$	$A \Rightarrow (B \Rightarrow C)$
T	T	T	T	T	T	T
T	T	T	F	F	F	F
T	F	F	T	T	T	T
T	F	F	F	T	T	T
F	T	T	T	T	T	T
F	T	T	F	F	F	T
F	F	T	T	T	T	T
F	F	T	F	F	T	T

⇒ is not associative.

ϕ: \phi
ψ: \psi

Implication

ϕ	ψ	$\phi \Rightarrow \psi$	$\sim\phi \vee \psi$
T	T	T	**T**
T	F	F	**F**
F	T	T	**T**
F	F	T	**T**

A helpful interpretation $\phi \Rightarrow \psi$ is "ψ is a logical consequence of ϕ".

We often view implication as if ϕ, then ψ.

Example.

Suppose I make a promise that, "*If I am healthy, I will go to class.*"

In each scenario do I keep my promise? Did I keep my promise?

1. I *am* healthy, and I *do* go to class. $(true \Rightarrow true)$ Yes.
2. I *am* healthy, and I *do not* go to class. $(true \Rightarrow false)$ No.
3. I *am not* healthy, and I *do* go to class. $(false \Rightarrow true)$ Yes.
4. I *am not* healthy, and I *do not* go to class. $(false \Rightarrow false)$ Yes.

$\phi \Rightarrow \psi$ would be false only if ϕ is true and ψ is false.

Inference

Knowledge Bases

How can an AI agent store, organize, and reason with everything it knows about the world?

Definition*.** A ***knowledge base KB is a set of sentences

$$KB = \{A_1, A_2, \dots, \mathrm{A}_n\}$$

These sentences are typically (conceptually) connected by logical conjunctions:

$$KB = A_1 \wedge A_2 \wedge \cdots \wedge \mathrm{A}_n.$$

Each sentence A_i represents a fact or assertion about the world, expressed in a formal logic language.

Knowledge Base Entailment and Inference

Entailment tells us what ***must be true*** given the facts and rules in the KB; it is the foundation for logical reasoning in AI systems.

Definition. In propositional logic, a sentence S is ***entailed*** by a knowledge base KB, written as

$$KB \vDash S$$

if and only if *S* is true in every model where all sentences in KB are true.

But, what else can we conclude from our KB?

Ultimately, our goal is to ***infer*** (discover) new sentences that logically follow from a knowledge base.

Knowledge Base Operations

Add new knowledge	TELL(KB, $sentence$)
Retrieve entailed information	ASK(KB, $query$)

Knowledge Base Operations: Example I

Suppose $KB = \{$ P: Alex is a student, S: Alex needs a textbook

Q: Alex is enrolled in CSC-343 $\}$

In propositional logic notation, we have $KB = \{P \wedge Q \wedge S\}$

TELL(KB, If Alex is enrolled in CSC-343, then Alex needs a textbook)

results in $KB = \{P \wedge Q \wedge S \wedge (Q \Rightarrow S)\}$

Knowledge Base Operations: Example II

Assume: $KB = \{P \wedge Q \wedge (Q \Rightarrow S)\}$

Since S is true in *every* model where KB is true (just this one model), we conclude:

$$KB \vDash S$$

$$\text{ASK}(KB,\ S) \ = \ \text{true}$$

P	Q	S	$Q \Rightarrow S$	$P \wedge Q \wedge (Q \Rightarrow S)$	S
T	T	T	T	T	T
T	T	F	F	F	F
T	F	T	T	F	T
T	F	F	T	F	F
F	T	T	T	F	T
F	T	F	F	F	F
F	F	T	T	F	T
F	F	F	T	F	F

Knowledge Base Operations: Example III

Assume: $KB = \{P \wedge Q\}$

$(Q \Rightarrow S)$ is ***not*** true in *every* model where KB is true, we conclude:

$$KB \nvDash (Q \Rightarrow S)$$

$$\text{ASK}(KB, Q \Rightarrow S) \quad = \quad \text{false}$$

P	Q	S	$Q \Rightarrow S$	$KB = P \wedge Q$
T	T	T	T	T
T	T	F	F	T
T	F	T	T	F
T	F	F	T	F
F	T	T	T	F
F	T	F	F	F
F	F	T	T	F
F	F	F	T	F

Knowledge Base Operations: Example IV

Assume: $KB = \{P \wedge Q \wedge S\}$

$(Q \Rightarrow S)$ is true in the only model where KB is true, we conclude:

$$KB \vDash (Q \Rightarrow S)$$

$$\text{ASK}(KB,\ Q \Rightarrow S) \quad = \quad \text{true}$$

P	Q	S	$KB = P \wedge Q \wedge S$	$Q \Rightarrow S$
T	T	T	T	T
T	T	F	F	F
T	F	T	F	T
T	F	F	F	T
F	T	T	F	T
F	T	F	F	F
F	F	T	F	T
F	F	F	F	T

Caution: Implication (⇒) does not capture causation. While KB technically entails $Q \Rightarrow S$ (because S is already true), this does not mean Q is the *reason* for S. Propositional logic cannot distinguish correlation from causation.

We cannot conclude enrollment *causes* the textbook need. Maybe Alex needs it for another course!

From Logic to Learning

The Limitation

Extracting patterns from instances is a task that propositional logic was not designed for.

Given only facts in a knowledge base, searching for rules that relate them is computationally expensive and yields only correlations, not meaningful knowledge.

Approach	Input	Output	Direction
Logic / Knowledge Bases	Rules + Facts	New Facts	Deduction
Machine Learning	Instances / Data	Patterns / Rules	Induction

The Bridge

This motivates ***machine learning***: given ground truth data, can we automatically induce the rules?

Neural networks and other ML methods address exactly this challenge: learning patterns from examples rather than being told the rules explicitly.

Inference

Inference is the process of deriving new sentences that are entailed by a KB. Fundamentally, it allows us to answer queries, make decisions, and automate reasoning.

Definition. Inference is the process of deriving a sentence S from a knowledge base KB, written as

$$KB \vdash S$$

if and only if there exists a way for S to be derived from KB using a *sound inference procedure**.

**we will talk about this*

From our prior example with $KB = \{P \wedge Q \wedge (Q \Rightarrow S)\}$, we can say $KB \vdash S$.

Given a knowledge base (KB), we want to derive new sentences that logically follow without checking every possible truth assignment (aka brute-force model checking).

Methods of Inference: Enumeration

Enumeration is an inference procedure that checks whether a sentence is entailed by a knowledge base by evaluating ***all possible truth assignments*** to the propositional symbols.

Suppose $KB = \{P \wedge Q \wedge (P \Rightarrow S)\}$ and ASK$(KB,\ S)$.

P	Q	S	$P \Rightarrow S$	$KB = \{P \wedge Q \wedge (P \Rightarrow S)\}$
T	T	T	T	T
T	T	F	F	F
T	F	T	T	F
T	F	F	F	F
F	T	T	T	F
F	T	F	T	F
F	F	T	T	F
F	F	F	T	F

$KB \vdash S$?

Yes.
There exists a way to derive S from KB.

$KB \vDash S$?

Yes.
Recall $KB \vDash S$ if and only if in every model where KB is true, S is also true.

Enumeration is an intractable procedure. Why?

2^n models for n variables.

Methods of Inference: Syntactic Manipulation

Inference rules provide precise, efficient steps for deriving conclusions: offering clarity and control (explainability) compared to broader procedures like enumeration or resolution, which are more exhaustive but less intuitive.

Modus Ponens

$A \Rightarrow B, A \quad \vdash B$

Premise 1: $A \Rightarrow B$

Premise 2: A

Conclusion: B

And-Elimination

$A \wedge B \quad \vdash A, B$

Premise 1: $A \wedge B$

Conclusion 1: A

Conclusion 2: B

DeMorgan's Laws

$\sim(A \wedge B) \equiv \sim(A) \vee \sim(B)$

$\sim(A \vee B) \equiv \sim(A) \wedge \sim(B)$

There are many others.

Logical Inference Example

Example.

Premises: $\sim(P \vee Q)$

$\sim P \Rightarrow R$

$R \Rightarrow S$

$\sim(P \vee Q) \equiv \sim P \wedge \sim Q$ DeMorgan's

$\Rightarrow \sim P \wedge \sim Q$

$\sim P \wedge \sim Q$ And-Elimination

$\Rightarrow \sim P$

$\Rightarrow \sim Q$

$\sim P, \sim P \Rightarrow R$ Modus Ponens

$\Rightarrow R$

$R, R \Rightarrow S$ Modus Ponens

$\Rightarrow S$

Modus Ponens

$A \Rightarrow B, A \quad \vdash B$

Premise 1: $A \Rightarrow B$

Premise 2: A

Conclusion: B

And-Elimination

$A \wedge B \quad \vdash A, B$

Premise 1: $A \wedge B$

Conclusion 1: A

Conclusion 2: B

DeMorgan's Laws

$\sim(A \wedge B) \equiv \sim(A) \vee \sim(B)$

$\sim(A \vee B) \equiv \sim(A) \wedge \sim(B)$

Other Rules of Logical Inference

Rule	Equivalence
Commutativity of ∧	$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$
Commutativity of ∨	$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$
Associativity of ∧	$(\alpha \wedge \beta) \wedge \gamma \equiv \alpha \wedge (\beta \wedge \gamma)$
Associativity of ∨	$(\alpha \vee \beta) \vee \gamma \equiv \alpha \vee (\beta \vee \gamma)$
Double-Negation Elimination	$\sim(\sim\alpha) \equiv \alpha$
Contraposition	$(\alpha \Rightarrow \beta) \equiv (\sim\beta \Rightarrow \alpha)$
Implication Elimination	$(\alpha \Rightarrow \beta) \equiv (\sim\alpha \vee \beta)$
Biconditional Elimination	$(\alpha \Leftrightarrow \beta) \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
DeMorgan	$\sim(\alpha \wedge \beta) \equiv (\sim\alpha \vee \sim\beta)$
DeMorgan	$\sim(\alpha \vee \beta) \equiv (\sim\alpha \wedge \sim\beta)$
Distributivity of ∧ over ∨	$\big(\alpha \wedge (\beta \vee \gamma)\big) \equiv \big((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)\big)$
Distributivity of ∨ over ∧	$\big(\alpha \vee (\beta \wedge \gamma)\big) \equiv \big((\alpha \vee \beta) \wedge (\alpha \vee \gamma)\big)$
Absorption	$\big(\alpha \wedge (\alpha \vee \beta)\big) \equiv \alpha \equiv \big(\alpha \vee (\alpha \wedge \beta)\big)$

Methods of Inference: Chaining

A rule-based system has the following components:

- **Facts**: A database of true statements (known information).
- **Rules**: “If-then” statements that encode domain knowledge.
- **Inference Engine**: Algorithms to apply rules to facts to derive new knowledge.

There are two main approaches in rule-based systems:

- Data-driven reasoning with ***Forward Chaining***.
- Goal-driven reasoning with ***Backward Chaining***.

Forward Chaining

(Data-driven reasoning.)

```
FORWARD_CHAINING(KB, query):
    facts = initial_facts
    if query in facts: return True

    new_facts = True
    while new_facts:
        new_facts = False
        for each rule in KB:
            if not rule.conditions_satisfied_by(facts):
                continue
            if rule.conclusion in facts:
                continue

            add rule.conclusion to facts
            new_facts = True
            if rule.conclusion == query: return True

    return False
```

Iterate exhaustively while new facts are inferred.

Find rules whose conditions match current facts

A new fact has been derived when all ‘conditions’ are satisfied by database facts.

Forward Chaining: Example

Facts	$\{A, B, C\}$		$\{A, B, C\}$		$\{A, B, C, D, E, G\}$		$\{A, B, C, D, E, F, G, H, J\}$		$\left\{\begin{matrix} A, B, C, D, E, F, \\ G, H, I, J, K \end{matrix}\right\}$
Rules									
R1: $A \wedge B \Rightarrow D$			$A \wedge B \Rightarrow D$	D					
R2: $C \Rightarrow E$			$C \Rightarrow E$	E					
R3: $D \wedge E \Rightarrow F$					$D \wedge E \Rightarrow F$	F			
R4: $A \wedge C \Rightarrow G$			$A \wedge C \Rightarrow G$	G					$\emptyset$
R5: $G \Rightarrow H$					$G \Rightarrow H$	H			
R6: $F \wedge H \Rightarrow I$							$F \wedge H \Rightarrow I$	I	
R7: $D \Rightarrow J$					$D \Rightarrow J$	J			
R8: $E \wedge J \Rightarrow K$							$E \wedge J \Rightarrow K$	K	

Observe that the forward chaining algorithm as defined, is inefficient. We could easily optimize it using a 'marking' algorithm which tracks 'used' rules. Thus, we would not need to iterate over those same rules repeatedly.

Forward Chaining: Try It

Facts $\{A, B, C\}$

Rules

R1: $A \wedge B \Rightarrow D$

R2: $B \wedge C \Rightarrow E$

R3: $A \Rightarrow F$

R4: $D \wedge F \Rightarrow G$

R5: $E \Rightarrow H$

R6: $C \wedge D \Rightarrow I$

R7: $G \wedge H \wedge I \Rightarrow J$

R8: $F \wedge J \Rightarrow K$

R9: $D \wedge L \Rightarrow M$

Backward Chaining

Think of the goal as being the root of a tree. Backward chaining is an inference technique that starts with a goal and builds that tree downward using the rules set. Ultimately, if the leaves of that tree are all in the known set of facts, we have deduced our goal from the knowledge base.

```
BACKWARD_CHAINING(KB, goal):
    if goal in facts: return True

    for each rule in KB:
        if rule.conclusion == goal:
            if PROVE_PREMISES(rule.conditions):
                return True
    return False

PROVE_PREMISES(premises):
    for each premise in premises:
        if not BACKWARD_CHAINING(KB, premise):
            return False
    return True
```

Base case: goal is known.

Does a rule's conclusion, match our current goal?

Check all premises of matching rule.

Must satisfy all premises for a rule to infer its conclusion.

Backward Chaining: Example

Facts $\{A, B, C\}$ **Goal** K

Rules

R1: $A \wedge B \Rightarrow D$

R2: $C \Rightarrow E$

R3: $D \wedge E \Rightarrow F$

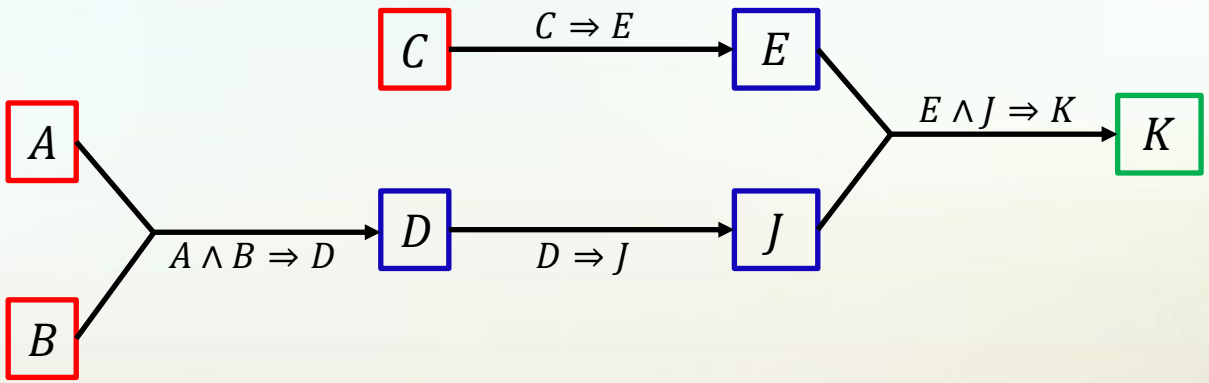
R4: $A \wedge C \Rightarrow G$

R5: $G \Rightarrow H$

R6: $F \wedge H \Rightarrow I$

R7: $D \Rightarrow J$

R8: $E \wedge J \Rightarrow K$



Backward chaining algorithm seems more efficient than forward training since it avoids unnecessary inference focusing on what is needed to prove the goal. However, it requires a starting point.

Backward Chaining: Try It

Facts $\{A, B, C\}$ **Goal** $\boxed{K}$

Rules

R1: $A \wedge B \Rightarrow D$

R2: $B \wedge C \Rightarrow E$

R3: $A \Rightarrow F$

R4: $D \wedge F \Rightarrow G$

R5: $E \Rightarrow H$

R6: $C \wedge D \Rightarrow I$

R7: $G \wedge H \wedge I \Rightarrow J$

R8: $F \wedge J \Rightarrow K$

R9: $D \wedge L \Rightarrow M$

Backward Chaining: Try It II

Facts $\{C, G\}$ **Goal** $\boxed{H}$

Rules

R1: $A \wedge B \Rightarrow D$

R2: $\mathrm{B} \wedge C \Rightarrow D$

R3: $E \wedge F \Rightarrow D$

R4: $D \wedge G \Rightarrow H$

R5: $H \Rightarrow K$

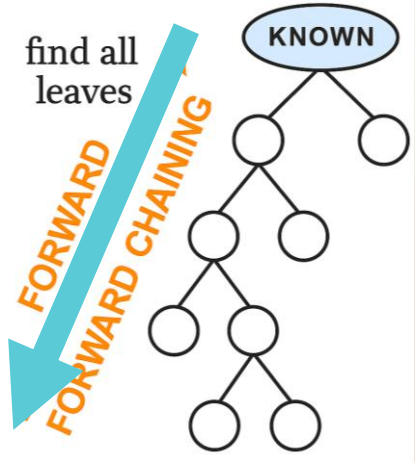
Chaining Analogies

Forward Chaining

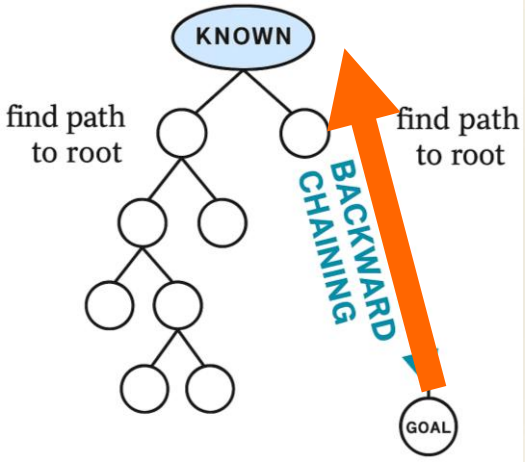
- Start from known facts as the root of a tree.
- Apply rules to derive all possible conclusions.
- Explores every branch of a tree en route to inferring any / all facts (leaves in the tree).
- Goal: discovering everything that can be inferred.

Backward Chaining

- Start from a leaf (goal or query).
- Backward hypergraph search: Work backward tracing a path from the goal leaf back to the root.
- Goal: inferring a specific fact.



Copilot generated images! Ug.



Resolution

A Motivating Example

Problem. Prove that there are infinitely many prime numbers.

Can we prove this statement directly?

Doing so would require that we list or characterize all prime numbers (e.g., p is of the form $2^n + 1$ for $n > 1$).

Instead…consider a short and elegant proof.

Proof by contradiction.

Suppose there are a finite number of prime numbers, enumerated as $p_1, p_2, \dots p_n$. Now, construct a value $N = p_1 \cdot p_2 \cdot \dots \cdot p_n + 1$. Clearly, N is not divisible by any prime p_i (as $N \% p_i = 1$ for all i).

So we can conclude that either (1) N is prime itself or (2) it is divisible by a prime number not in the list. In either case, we have a contradiction since all primes were in our enumerated list $p_1, p_2, \dots p_n$. ■

Methods of Inference: Resolution

Recall with proof by contraction, when proving $p \Rightarrow q$, we are instead attempting to prove:

$$p \wedge \sim q \Rightarrow false$$

p	q	$p \Rightarrow q$	$p \wedge \sim q$
T	T	T	F
T	F	F	T
F	T	T	F
F	F	T	F

Resolution, as an inference procedure, uses the idea of proof by contradiction.

(The term refutation is often used in logic programming because the method focuses on refuting the negation of the goal.)

Instead of juggling multiple inference rules, resolution uses *one uniform rule* to derive contradictions.

Direct inference $\quad KB \vDash P \quad \Leftrightarrow \quad \boxed{KB \wedge \sim P \vDash false}$

Resolution via Refutation

Conjunctive Normal Form (CNF)

A ***clause*** is a disjunction of literals (where a literal is an atomic proposition or its negation) whereas a ***sentence in CNF*** is a conjunction of one or more clauses.

$$\underbrace{\underbrace{(\sim A \vee B \vee C)}_{\text{clause}} \wedge \underbrace{(\sim B \vee A)}_{\text{clause}} \wedge \underbrace{(A \vee \sim C)}_{\text{clause}}}_{\text{CNF}}$$

Converting to CNF

A ***clause*** is a disjunction of literals.
A ***sentence in CNF*** is a conjunction of one or more clauses.

$$\underbrace{\underbrace{(\sim A \vee B \vee C)}_{\text{clause}} \wedge \underbrace{(\sim B \vee A)}_{\text{clause}} \wedge \underbrace{(A \vee \sim C)}_{\text{clause}}}_{\text{CNF}}$$

Input: A sentence S.

Output: A sentence in CNF.

1. Eliminate implications: $\Rightarrow$ and $\Leftrightarrow$.
2. Move negation ($\sim$) inward.
3. Apply distributive laws: $\vee$ over $\wedge$.
4. Remove duplicate literals in a clause.
5. Remove tautologies.

Distributivity of $\vee$ over $\wedge$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$$

Try It

$$\sim\big((P \Rightarrow Q) \wedge (R \Rightarrow S)\big)$$

1. Eliminate implications: $\Rightarrow$ and $\Leftrightarrow$.
2. Move negation ($\sim$) inward.
3. Apply distributive laws: $\vee$ over $\wedge$.
4. Remove duplicate literals in a clause.
5. Remove tautologies.

Distributivity of $\vee$ over $\wedge$

$$\big(\alpha \vee (\beta \wedge \gamma)\big) \equiv \big((\alpha \vee \beta) \wedge (\alpha \vee \gamma)\big)$$

$$(P \vee R) \wedge (P \vee \sim S) \wedge (\sim Q \vee R) \wedge (\sim Q \vee \sim S)$$

Example: Converting to CNF I

1. Eliminate implications: $\Rightarrow$ and $\Leftrightarrow$.
2. Move negation ($\sim$) inward.
3. Apply distributive laws: $\vee$ over $\wedge$.
4. Remove duplicate literals in a clause.
5. Remove tautologies.

Distributivity of $\vee$ over $\wedge$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$$

$$\sim(P \Rightarrow (Q \Leftrightarrow R)) \vee S$$

Eliminate $\Leftrightarrow$

$$\Leftrightarrow \quad \sim\big(P \Rightarrow ((Q \Rightarrow R) \wedge (R \Rightarrow Q))\big) \vee S$$

Eliminate $\Rightarrow$

$$\Leftrightarrow \quad \sim\big(\sim P \vee ((\sim Q \vee R) \wedge (\sim R \vee Q))\big) \vee S$$

Move negation ($\sim$) inward

$$\Leftrightarrow \quad \big(P \wedge \sim((\sim Q \vee R) \wedge (\sim R \vee Q))\big) \vee S$$

Move negation ($\sim$) inward

$$\Leftrightarrow \quad \big(P \wedge (\sim(\sim Q \vee R) \vee \sim(\sim R \vee Q))\big) \vee S$$

Move negation ($\sim$) inward

$$\Leftrightarrow \quad \big(P \wedge ((Q \wedge \sim R) \vee (R \wedge \sim Q))\big) \vee S$$

Let X be $(Q \wedge \sim R) \vee (R \wedge \sim Q)$

$$\Leftrightarrow \quad (P \wedge X) \vee S$$

Apply distributive laws: $\vee$ over $\wedge$

$$\Leftrightarrow \quad (P \vee S) \wedge (X \vee S)$$

Remember to link expressions

Example: Converting to CNF II

1. Eliminate implications: $\Rightarrow$ and $\Leftrightarrow$.
2. Move negation ($\sim$) inward.
3. Apply distributive laws: $\vee$ over $\wedge$.
4. Remove duplicate literals in a clause.
5. Remove tautologies.

Distributivity of $\vee$ over $\wedge$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$$

$$(P \vee S) \wedge (X \vee S)$$

Handle X:

$$X \equiv (Q \wedge \sim R) \vee (R \wedge \sim Q)$$

Apply distributive laws: $\vee$ over $\wedge$

$$\Leftrightarrow \quad (Q \vee (R \wedge \sim Q)) \wedge (\sim R \vee (R \wedge \sim Q))$$

Apply distributive laws: $\vee$ over $\wedge$

$$\Leftrightarrow \quad ((Q \vee R) \wedge (Q \vee \sim Q)) \wedge ((\sim R \vee R) \wedge (\sim R \vee \sim Q))$$

Associativity of $\wedge$

$$\Leftrightarrow \quad (Q \vee R) \wedge (Q \vee \sim Q) \wedge (\sim R \vee R) \wedge (\sim R \vee \sim Q)$$

Remove Tautologies

$$\Leftrightarrow \quad (Q \vee R) \wedge (\sim R \vee \sim Q)$$

$$\Leftrightarrow \quad (P \vee S) \wedge (((Q \vee R) \wedge (\sim R \vee \sim Q)) \vee S)$$

Apply distributive laws: $\vee$ over $\wedge$

$$\Leftrightarrow \quad (P \vee S) \wedge ((Q \vee R \vee S) \wedge (\sim R \vee \sim Q \vee S))$$

Associativity of $\wedge$

$$\Leftrightarrow \quad \boxed{(P \vee S) \wedge (Q \vee R \vee S) \wedge (\sim R \vee \sim Q \vee S)}$$

The Resolution Rule

Suppose we have clauses A, L, and B.

$$\frac{(A \vee L) \wedge (\sim L \vee B)}{(A \vee B)}$$

parent clauses

resolvent

From a truth table:

A	L	B	$A \vee L$	$\sim L \vee B$	$A \vee B$
T	T	T	T	T	T
T	T	F	T	F	T
T	F	T	T	T	T
T	F	F	T	T	T
F	T	T	T	T	T
F	T	F	T	F	F
F	F	T	F	T	T
F	F	F	F	T	F

Whenever the parent clauses are both true, the resolvent is also true.

This is proof that the Resolution rule is ***sound***: the resolvent is a logical consequence of its parent clauses.

Resolution Procedure

Input:

- Knowledge base: KB
- Query sentence: Q

1) Combine $KB \wedge \sim Q$.
2) Rename variables to avoid clashes.
3) Convert $KB \wedge \sim Q$ to CNF.
4) Apply the Resolution rule.
5) Repeat (4) until either:
   - Empty clause ($\perp$) is derived →
     - Contradiction: $KB \wedge \sim Q$ is unsatisfiable
     - $KB \vDash Q$.
   - No new clauses can be generated →
     - $KB \nvDash Q$ (Goal cannot be derived from KB.)

$$\frac{(A \vee L) \wedge (\sim L \vee B)}{(A \vee B)}$$

Iterating: You are basically trying to prove the system breaks: if it does, your original idea was right.

Iterating: It's like solving a puzzle: keep combining pieces until you either finish it or realize some pieces are missing.

Resolution Procedure Example

1. Combine $KB \wedge \sim Q$.
2. Rename variables to avoid clashes.
3. Convert $KB \wedge \sim Q$ to CNF.
4. Apply the Resolution rule.
5. Repeat until ($\perp$) or exhaustion

$$\frac{(A \vee L) \wedge (\sim L \vee B)}{(A \vee B)}$$

Input:

- Knowledge base: $\{P \vee Q, \sim P \vee R, \sim Q \vee R\}$
- Query sentence: R

The clauses are in CNF, so we have $KB \wedge \sim R \equiv$

$$(P \vee \mathrm{Q}) \wedge (\sim P \vee R) \wedge (\sim Q \vee R) \wedge (\sim R)$$

Repeatedly apply the Resolution Rule to the existing KB:

$$\frac{(\sim Q \vee R) \wedge (\sim R)}{\sim Q} \qquad A = A \vee false$$

$$\frac{(P \vee \mathrm{Q}) \wedge (\sim P \vee R)}{(Q \vee R)}$$

$$\frac{(P \vee \mathrm{Q}) \wedge (\sim Q \vee R)}{(P \vee R)}$$

Using the computed clauses:

$$\frac{(P \vee Q) \wedge (\sim Q)}{P}$$

$$\frac{(Q \vee R) \wedge (\sim R)}{Q}$$

We observe that we derived $\mathrm{Q} \wedge \sim Q \vdash \perp$.

We conclude $KB \vDash R$.

Resolution Example II

1. Combine $KB \wedge \sim Q$.
2. Rename variables to avoid clashes.
3. Convert $KB \wedge \sim Q$ to CNF.
4. Apply the Resolution rule.
5. Repeat until ($\perp$) or exhaustion

$$\frac{(A \vee L) \wedge (\sim L \vee B)}{(A \vee B)}$$

Input:

- Knowledge base: $\{A \vee B \vee C, \sim A \vee D, \sim B \vee D, \sim C \vee \sim D\}$
- Query sentence: D

The clauses are in CNF, so we have $KB \wedge \sim D \equiv$

$$(A \vee B \vee C) \wedge (\sim A \vee D) \wedge (\sim B \vee D) \wedge (\sim C \vee \sim D) \wedge (\sim D)$$

Repeatedly apply the Resolution Rule to the existing KB:

$$\frac{(\sim A \vee D) \wedge (\sim D)}{\sim A} \qquad \frac{(A \vee B \vee \mathrm{C}) \wedge (\sim A)}{(B \vee C)} \qquad \frac{(\sim B \vee D) \wedge (\sim D)}{\sim B}$$

$$\frac{(B \vee C) \wedge (\sim B)}{C} \qquad \frac{(\sim C \vee \sim D) \wedge (C)}{\sim D}$$

We have exhausted the useful resolutions.

We conclude $KB \nvDash D$.

Soundness and Completeness

Summary

Soundness	*"Don't derive false conclusions."*	$KB \vdash \phi \Rightarrow KB \vDash \phi$
Completeness	*"Can derive all true conclusions ."*	$KB \vDash \phi \Rightarrow KB \vdash \phi$

Goal

A **Sound** and **Complete** Inference System

Why?

Syntactic derivation ⇔ Semantic validity

Syntactic ($KB \vdash \phi$): Rule application, symbol manipulation

Semantic ($KB \vDash \phi$): Truth-based reasoning about logical relationships

In a sound and complete system, both approaches always give the same answer.

Soundness (of Propositional Logic)

Our general definition:

Definition. An argument is ***sound*** if it is valid *and* all premises are actually true.

Soundness is a guarantee that our inference system never leads us astray: every derivable conclusion is logically valid.

Definition. An inference system is ***sound*** if and only if for all knowledge bases KB and all sentences ϕ,

$$KB \vdash \phi \Rightarrow KB \vDash \phi$$

(If ϕ is derivable from KB using the inference rules, then ϕ is semantically entailed by KB.)

Without a sound inference system, we cannot trust our proofs since we might derive ϕ from $\sim\phi$.

Example: Not Sound

Suppose we accidentally design a system that includes an ***invalid rule***:

From $P \vee Q$ we can derive P.

$$\frac{(P \vee Q)}{P}$$

Consider a knowledge base $KB = \{R, R \Rightarrow (S \vee T)\}$.

Using modus ponens, $KB = \{R, R \Rightarrow (S \vee T)\}$, we can derive $S \vee T$.

$$\frac{R, R \Rightarrow (S \vee T)}{S \vee T}$$

Using our ***invalid rule***, we derive S. $\boxed{\text{That is, } KB \vdash S.}$ But to verify soundness, we need to show $KB \vDash S$.

Recall, sound systems require that every inference rule preserves truth in all models.

Suppose $R = true$, $S = false$, and $T = true$. Then,

✓ $R = true$

✓ $R \Rightarrow (S \vee T)$ $\quad true \Rightarrow (false \vee true) \quad \Leftrightarrow \quad true \Rightarrow true \quad \Leftrightarrow \quad true$

✓ $S \vee T$ $\quad false \vee true \quad \Leftrightarrow \quad true$

✗ $S = false$

S is not semantically entailed: $KB \nvDash S$.

Completeness (of Propositional Logic)

Completeness is a guarantee that our inference system can find all valid conclusions (i.e., no 'missing' logical consequences).

Definition. An inference system is ***complete*** if and only if for all knowledge bases KB and all sentences ϕ,

$$KB \vDash \phi \Rightarrow KB \vdash \phi$$

(If ϕ is semantically entailed by KB, then ϕ is derivable from KB using the inference rules.)

Without a complete inference system, some valid conclusions might be unprovable (since we would miss important logical consequences).

Example: Not Complete

Suppose we have an inference system that only includes one inference rule: modus ponens.

From $P \Rightarrow Q$ and P, we can derive Q.

$$\frac{(P \Rightarrow Q) \wedge P}{Q}$$

Consider a knowledge base $KB = \{R \vee S, \sim R\}$ with goal S.

Definition [semantic entailment]. $KB \vDash Q$ means "In every model where KB is true, Q is also true."

We first perform a semantic check: $KB \vDash S$.

$R = true$. Then, this is a contradiction as $KB \vDash \sim R$; KB cannot also entail R.

$R = false$. Then, from $R \vee S$, we know S must be true.

$\boxed{\text{Thus, } KB \vDash S.}$

Now, can we derive S? $KB \vdash S$?

Modus ponens requires a conditional format: $P \Rightarrow Q$. Our knowledge base does not have any clauses of this form.

Therefore, $KB \nvdash S$.

Thus, this inference system is not complete.

Theorem. Resolution for propositional logic is both sound and complete.

Theorem. If $KB \wedge \sim\phi \vdash \perp$ by Resolution, then $KB \vDash \phi$.

Proof. Suppose Resolution derives $\perp$ from $KB \wedge \sim\phi$.

First, we establish that the resolution rule is truth-preserving.

Define two clauses $C_1 = (L \vee A)$ and $C_2 = (\sim L \vee B)$ that are both satisfiable and also let $R = (A \vee B)$ be their resolvent.

Let M be a model that satisfies both C_1 and C_2. Then M must assign some truth value to L.

If $M(L) = true$, then since M satisfies $C_2 = (\sim L \vee B)$, we must have $M(B) = true$, which makes $M(R) = M(A \vee B) = true$.

If $M(L) = false$, then since M satisfies $C_1 = (L \vee A)$, we must have $M(A) = true$, which again makes $M(R) = M(A \vee B) = true$.

Therefore, R is satisfiable whenever both parent clauses are satisfiable.

Since resolution preserves satisfiability, any clause derived through resolution can only be satisfiable if the original clauses are satisfiable. Therefore, if we derive $\perp$ (the empty clause, which is always unsatisfiable), from $KB \wedge \sim\phi$, this means $KB \wedge \sim\phi$ must be unsatisfiable.

That is, there is no model that makes both KB and $\sim\phi$ simultaneously true. Thus, in every model where KB is true, $\sim\phi$ must be false. Hence, ϕ must be true. It follows $KB \vDash \phi$. ■

Theorem. If $KB \vDash \phi$, then Resolution derives $\perp$ from $KB \wedge \sim\phi$.

Proof. Suppose $KB \vDash \phi$. Then $KB \wedge \sim\phi$ is unsatisfiable.

Now, assume for the sake of contradiction that Resolution does not derive $\perp$ from $KB \wedge \sim\phi$.

Observe that for propositional logic there are finitely many distinct clauses. Thus, we know that Resolution terminates with a finite clause set S.

Since Resolution did not derive $\perp$, the final clause set S must be satisfiable. We therefore can construct a truth assignment that satisfies all clauses in S:

- For each propositional variable P, if there exists a unit clause P in S, assign $P = true$;
- If there exists a unit clause $\sim P$ in S, assign $P = false$;
- Otherwise, assign P arbitrarily.

Recall the Resolution procedure begins with $KB \wedge \sim\phi$. Since S contains all clauses from the CNF conversion of $KB \wedge \sim\phi$ (as well as possibly other derived clauses), the truth assignment we constructed also satisfies $KB \wedge \sim\phi$. This is a contradiction.

Therefore, Resolution must derive $\perp$. ■

Summary (Again)

Soundness *"Don't derive false conclusions."* $KB \vdash \phi \Rightarrow KB \vDash \phi$

In practice: No false positives – the inference system will not "hallucinate" false conclusions.

Real-World Example: If a medical diagnosis system says "a patient has cancer," we want to be confident it is correct.

Completeness *"Can derive all true conclusions ."* $KB \vDash \phi \Rightarrow KB \vdash \phi$

In practice: No false negatives – the system will not miss valid conclusions.

Real-World Example: If a patient actually has cancer (and this entailed by the symptoms), we want our system to identify it.

Our Goal: Sound and Complete systems.
They ensure our AI is both trustworthy (no false conclusions) and comprehensive (no missed inferences).